# Manual Técnico: Sistema de Gestión Académica "AcadNet"

## 1. Descripción General del Sistema

**AcadNet** es una plataforma distribuida orientada a la gestión eficiente de asignaciones y calificaciones estudiantiles. El núcleo del sistema está diseñado para optimizar el consumo de memoria RAM mediante el uso de estructuras de datos no lineales, específicamente una **Matriz Dispersa Ortogonal**.

## 2. Arquitectura de Microservicios

El proyecto divide sus responsabilidades en dos entornos independientes que se comunican mediante peticiones HTTP (API REST):

- **Backend (Motor Lógico - Flask):** Encargado de la persistencia de datos en memoria, la validación de archivos XML y el manejo de la estructura de la matriz. Se ejecuta en el puerto 5000.
- **Frontend (Interfaz de Usuario - Django):** Responsable de la renderización de vistas, gestión de sesiones y recolección de datos del usuario final. Se ejecuta en el puerto 8000.

## 3. Estructura de Datos Principal

Para solventar el problema de la densidad de datos (estudiantes sin asignaciones en todos los cursos), se prescindió de las matrices densas tradicionales O(N x M) a favor de una **Matriz Dispersa**.

### 3.1. Clase Nodo

Representa la intersección entre un estudiante y una actividad.

- **Atributos de Datos:** id_fila (Actividad), id_columna (Carnet) y valor (Nota).
- **Punteros:** Implementa cuatro enlaces direccionales (arriba, abajo, izquierda, derecha) para permitir recorridos ortogonales.

### 3.2. Clase MatrizDispersa

Administra el ciclo de vida de los Nodos.

- **Cabeceras Indexadas:** Utiliza diccionarios nativos de Python (dict) para almacenar las cabeceras de filas y columnas. Esto garantiza un tiempo de acceso O(1) para localizar el inicio de cualquier recorrido, optimizando las consultas.

```python
class Nodo:
    def __init__(self, fila: str, columna: str, valor: float):
        self.fila = fila
        self.columna = columna
        self.valor = valor
        self.siguiente_fila = None    # Nodo a la derecha
        self.siguiente_columna = None # Nodo abajo

class MatrizDispersa:
    def __init__(self, nombre_curso: str):
        self.nombre_curso = nombre_curso
        self._filas = {}      # Diccionario de cabeceras de filas
        self._columnas = {}   # Diccionario de cabeceras de columnas
        self._orden_filas = []
        self._orden_columnas = []

    def insertar(self, fila: str, columna: str, valor: float) -> None:
        if valor < 0 or valor > 100:
            return
```

---

## 4. Diagrama de Clases del Motor Lógico

*(Nota para ti: Aquí debes pegar la imagen del diagrama UML con la relación de Composición que generamos anteriormente. Asegúrate de ponerle el título: "Figura 1: Estructura UML de la Matriz Dispersa y sus Nodos").*

---

## 5. Diccionario de Endpoints (API REST)

El Backend expone los siguientes servicios para la integración con la interfaz:

| Método | Endpoint | Descripción | Payload / Respuesta |
|---|---|---|---|
| **POST** | /api/admin/procesar | Inicializa las cabeceras y valida la integridad de estudiantes, tutores y cursos. | Recibe archivo XML de configuración. |

| Método | Endpoint | Descripción | Payload / Respuesta |
|---|---|---|---|
| **GET** | /api/admin/generar-reporte | Retorna el consolidado de asignaciones exitosas y rechazadas. | Devuelve archivo XML estructurado. |
| **POST** | /api/tutor/inyectar-notas | Instancia los Nodos en la matriz con las calificaciones correspondientes. | Recibe archivo XML de notas. |
| **GET** | /api/estudiante/consultar/<carnet> | Recorre la matriz verticalmente para extraer el historial de un alumno. | Devuelve arreglo JSON con actividades y notas. |

**6. Lógica de Validación (Integridad Referencial)**

El sistema cuenta con un validador estricto. Al momento de cargar una asignación, el flujo de ejecución es el siguiente:

1. Se extrae el codigo_curso y el carnet de la etiqueta XML.
2. Se verifica su existencia en las cabeceras previamente cargadas.
3. Si ambos existen, se procesa la inserción en la Matriz Dispersa.
4. Si alguno no existe, la petición se rechaza, asegurando que no se generen Nodos huérfanos, y se incrementa el contador de <incorrecto> para el reporte final.

**7. Requerimientos e Instalación**

Para desplegar el entorno de desarrollo local, se requiere:

- Python 3.13 o superior.
- Gestor de paquetes pip.

Comandos de inicialización:

```
# Servidor Backend
cd Backend
pip install flask flask-cors
python app.py

# Servidor Frontend
cd Frontend/acadnet_project
pip install django requests
python manage.py runserver
```
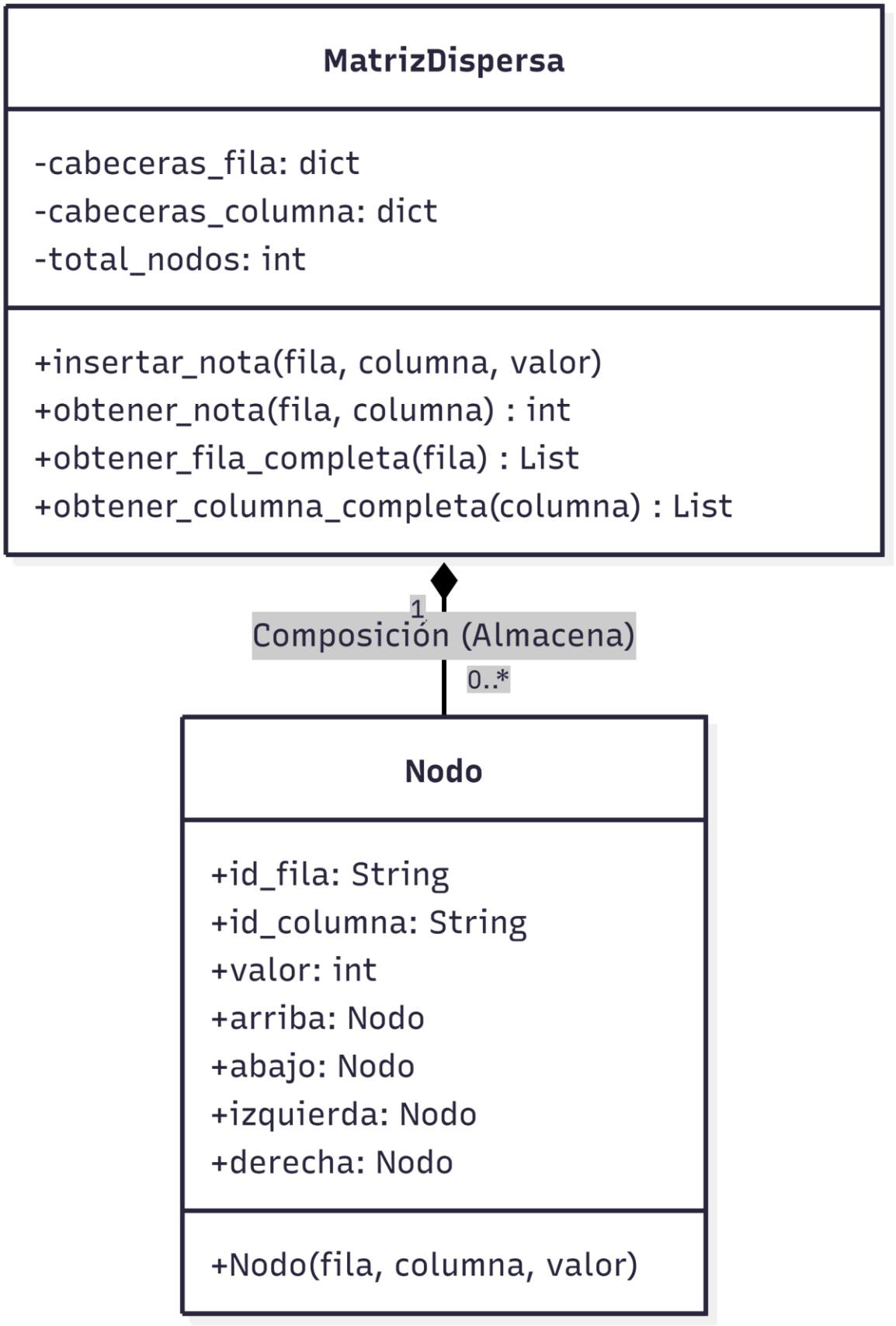
**Diagrama de clases**